

FPGA Implementation of an Artificial Neural Network Processor with Customized Instruction Set for ECG Signal Classification

Anonymous CVPR submission

Paper ID 14

Abstract

Electrocardiogram (ECG) signals provide important information about the electrical activity of the heart and can be used for detecting abnormal cardiac rhythms. Artificial neural networks (ANNs) have been investigated for automated ECG classification, while field-programmable gate arrays (FPGAs) provide a reconfigurable platform for hardware-based signal processing. This project proposes the implementation of an ANN processor on an FPGA for ECG signal classification using a customized instruction set architecture (ISA). The ANN will be trained offline using ECG data, after which its trained weights and biases will be converted into a fixed-point representation suitable for hardware implementation. A customized set of instructions will be developed to control operations such as loading data and parameters, performing multiply-accumulate operations, applying activation functions, and storing results. The proposed approach combines ECG classification with a programmable ANN processing architecture and provides a basis for implementing neural-network inference on FPGA hardware.

1. Introduction

Electrocardiography (ECG) is a commonly used technique for observing the electrical activity of the heart. ECG recordings contain characteristic waveforms associated with cardiac activity, and changes in their morphology can indicate abnormal cardiac conditions. Automated analysis of ECG signals can assist in identifying abnormal beats and reduce the effort required for manual examination.

Machine learning and artificial neural networks (ANNs) have been explored for ECG classification. Several studies have demonstrated neural-network-based ECG or arrhythmia classification, including implementations targeting FPGA hardware [1, 3, 4, 6]. FPGAs are attractive for neural-network inference because their configurable hardware resources can be used for application-specific compu-

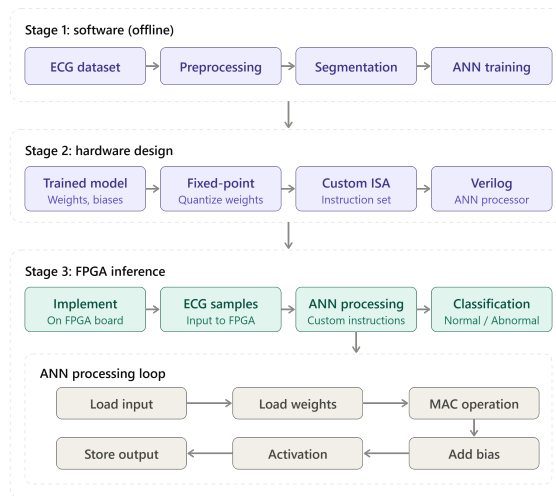


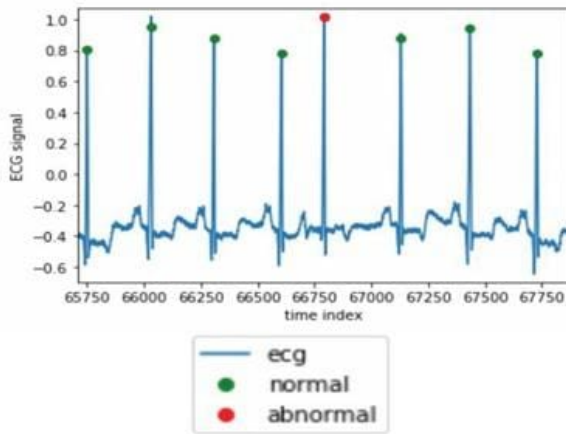
Figure 1. Overall proposed workflow for FPGA-based ECG signal classification.

tation.

A second direction relevant to this project is the use of customized instruction sets for ANN computation. Valencia *et al.* proposed a programmable ANN processor with a custom instruction set architecture for embedded applications [5]. Bagal and Harpale investigated an FPGA implementation of an ANN using a customized instruction set for one-dimensional signal processing [2]. These works motivate the use of a programmable processor architecture instead of implementing the complete neural network as one fixed datapath.

The proposed project combines these directions for ECG signal classification. The ANN will be trained in software and the trained model will subsequently be implemented for inference on an FPGA-based ANN processor controlled by a customized instruction set.

Problem Statement: The objective of this project is to develop a compact FPGA-based ANN processor capable of



performing ECG beat processing and classification using a customized instruction set. The system will process fixed-length ECG segments, execute the trained ANN using fixed-point arithmetic, and provide a binary Normal/Abnormal classification.

2. Literature Review

Research relevant to the proposed project can be grouped into four closely related areas: ANN implementation on FPGA, fixed-point neural-network computation, customized instruction-set architectures, and FPGA-based ECG classification.

2.1. ANN Implementation on FPGA

Won presented a hardware implementation of artificial neural networks using FPGAs and discussed the representation of trained neural-network parameters for digital hardware implementation [7]. The work provides a foundation for understanding how feed-forward ANN computation can be mapped to FPGA resources. Considerations such as finite-width numerical representation and hardware realization of activation functions are relevant to the proposed implementation.

2.2. ANN Processor with a Customized Instruction Set

Valencia, Fouladi Fard, and Alimohammad proposed an ANN processor with a custom instruction set architecture for embedded applications [5]. The architecture uses programmable instructions to control neural-network operations and supports configurable network structures. This work is particularly relevant because it demonstrates how ANN computation can be organized as processor instructions rather than being tied to one fixed neural-network implementation.

Bagal and Harpale [2] describe an FPGA implementation of an ANN with a customized instruction set for one-

dimensional signal processing, and evaluate ECG classification in software. However, the instruction set itself is not specified in detail, and no hardware results (resource utilization, latency, or power) are reported. This work is therefore used here as motivation, and its gaps (an explicit ISA definition and hardware-level verification) are addressed in the present project.

2.3. FPGA-Based ECG Classification

Vinaykumar and Thilagavathy investigated FPGA implementation of an ANN for ECG signal classification [6]. Their work provides a direct reference for implementing ANN-based ECG classification on FPGA hardware.

Dal and Askar studied fixed-point FPGA implementation of ECG classification using an artificial neural network [3]. Their work is relevant to the proposed fixed-point implementation because the trained neural-network computation must be represented in a hardware-compatible numerical format.

Srivastava *et al.* presented an FPGA implementation for automatic arrhythmia detection based on a probabilistic neural network [4]. This demonstrates another neural-network-based approach to ECG/arrhythmia classification on FPGA hardware.

Aruna *et al.* investigated acceleration of a deep convolutional neural network on FPGA for ECG signal classification [1]. This work represents a more complex neural-network architecture and illustrates the broader use of FPGA acceleration for ECG classification.

2.4. Research Motivation

The reviewed studies demonstrate that FPGA hardware can be used for ANN-based ECG classification and that fixed-point arithmetic is important for practical hardware implementation. The literature also demonstrates that customized instruction sets can provide programmability and flexibility for ANN processors.

Based on these directions, the proposed project focuses on combining ECG classification with a programmable ANN processor architecture. The intended workflow is to train the classifier offline, convert its parameters to a hardware-compatible fixed-point representation, and execute inference on an FPGA using a customized instruction sequence.

3. Next Steps

Proposed Approach

The following stages are planned for the continuation of the project.

3.1. ECG Dataset and Preprocessing

The MIT-BIH Arrhythmia Database will be used as the ECG dataset. The ECG recordings will be processed to

obtain fixed-length signal segments suitable for ANN input. For the preliminary implementation, a segment length of 180 samples is being considered. The dataset will be divided into training, validation, and testing subsets while maintaining separation between records to reduce data leakage.

The preprocessing stage will produce normalized fixed-length ECG vectors. The exact filtering, normalization, segmentation, and labeling procedure will be finalized during the software experiments.

3.2. ANN Training

The ANN classifier will first be developed and trained in software. The initial network being considered consists of an input layer corresponding to the ECG segment, a small hidden layer with a nonlinear activation function, and an output layer for binary classification. The network structure can be adjusted based on software-level evaluation before hardware implementation.

For a neuron in a fully connected layer, the weighted sum can be written as

$$z_j = \sum_{i=1}^N w_{ji}x_i + b_j, \quad (1)$$

where x_i denotes an input sample, w_{ji} denotes the corresponding weight, and b_j denotes the bias.

For the initial hidden layer, the ReLU activation is

$$h_j = \text{ReLU}(z_j) = \max(0, z_j). \quad (2)$$

For binary classification, the initial output formulation is a sigmoid function

$$\hat{y} = \sigma(z_o) = \frac{1}{1 + e^{-z_o}}. \quad (3)$$

The classification threshold can initially be defined as

$$\text{Class} = \begin{cases} \text{Normal}, & \hat{y} < 0.5, \\ \text{Abnormal}, & \hat{y} \geq 0.5. \end{cases} \quad (4)$$

These equations describe the initial model under consideration; the final architecture and thresholding procedure will be determined after software-level evaluation.

3.3. Fixed-Point Conversion

After selecting and training the ANN, its weights and biases will be converted from floating-point representation to a fixed-point format suitable for FPGA arithmetic. The effect of quantization on classification performance will be evaluated before the hardware implementation.

A simple fixed-point representation with F fractional bits can be expressed as

$$X_{\text{fixed}} = \text{round}(X_{\text{float}}2^F). \quad (5)$$

The corresponding approximate floating-point value is

$$X_{\text{float}} \approx \frac{X_{\text{fixed}}}{2^F}. \quad (6)$$

The quantization error can be monitored using

$$E_q = X_{\text{float}} - \frac{X_{\text{fixed}}}{2^F}. \quad (7)$$

The fixed-point representation will be selected by considering numerical range, required precision, accumulator width, and available FPGA resources. The same representation and scaling used during software verification will subsequently be used in the Verilog implementation.

3.4. Customized Instruction Set

A customized instruction set will be designed to represent the main operations required for ANN inference. The preliminary instruction categories include loading input or parameters, performing multiply-accumulate operations, adding biases, applying activation functions, and storing outputs.

A conceptual instruction sequence is

LOAD $\rightarrow$ MAC $\rightarrow$ BIAS $\rightarrow$ ACTIVATE $\rightarrow$ STORE.

The instruction set will be refined after the ANN architecture and fixed-point representation are finalized. The objective is to make the processor sufficiently programmable to execute the selected ANN while keeping the instruction format and hardware implementation manageable.

3.5. ANN Processor Architecture

The proposed processor will contain instruction memory, an instruction decoder, data/parameter memory, registers, a multiply-accumulate (MAC) unit, activation-function logic, and output storage. A program counter will control instruction execution.

The central hardware operation is the MAC operation

$$S_k = S_{k-1} + x_k w_k, \quad S_0 = b2^F, \quad (8)$$

where x_k and w_k are the input and weight values, respectively, and b is the bias. The accumulator width is

$$B_{\text{acc}} = 2B + \lceil \log_2 N \rceil \quad (9)$$

bits for B -bit operands and N products per neuron, i.e. 40 bits for $B = 16$ and $N = 180$, so overflow cannot occur within a layer. This also fits in the 48-bit accumulator of a Spartan-7 DSP slice. The final sum is shifted right by F bits and saturated to the B -bit range $[-2^{B-1}, 2^{B-1} - 1]$ before being passed to the activation stage. This operation forms the main computational kernel of the fully connected ANN layers.

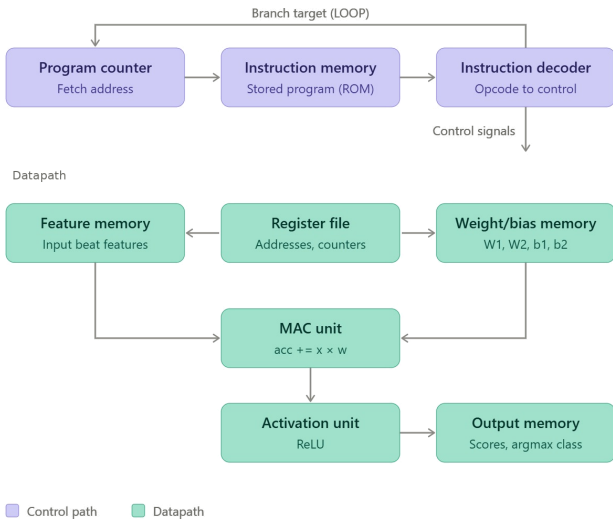


Figure 2. Conceptual architecture of the proposed customized-ISA ANN processor.

3.6. Verilog Implementation and FPGA Verification

The processor modules will be implemented in Verilog and verified through simulation. The software fixed-point model will be used as a reference for checking the hardware arithmetic and classification output.

Verification will be performed progressively. Individual modules such as the MAC unit, activation logic, register/memory interfaces, and instruction decoder will first be tested independently. The modules will then be integrated into the processor and tested using the selected instruction sequence.

After functional verification, the complete processor will be synthesized and implemented on the target FPGA board. The final stage will demonstrate execution of ANN inference using the customized instruction sequence and provide a Normal/Abnormal classification output.

3.7. Final System Integration

The final system will integrate the ECG preprocessing assumptions, trained ANN parameters, fixed-point representation, instruction memory, processor datapath, and FPGA interface. The hardware output will be compared against the software reference to verify correct operation.

The main comparison will be performed between the floating-point software model, the fixed-point software model, and the FPGA implementation. This will allow the effects of fixed-point conversion and hardware implementation to be evaluated before the final demonstration.

AI Usage Declaration

AI-based tools were used during the preparation of this report for language refinement, organization, figure prepa-

ration, and assistance in formatting the document. The technical design, project decisions, implementation, and experimental work remain the responsibility of the project team.

References

- [1] V. B. K. L. Aruna, E. Chitra, and M. Padmaja, "Accelerating deep convolutional neural network on FPGA for ECG signal classification," *Microprocessors and Microsystems*, vol. 103, p. 104939, 2023. 1, 2
- [2] A. Bagal and V. Harpale, "FPGA implementation of ANN using customized instruction set for 1-D signal processing," in *2022 International Conference on Computing, Communication, Control and Automation (ICCUBEA)*, 2022. 1, 2
- [3] B. Dal and M. Askar, "Fixed-point FPGA implementation of ECG classification using artificial neural network," in *2022 Medical Technologies Congress (TIPTEKNO)*, pp. 1–4, 2022. 1, 2
- [4] R. Srivastava, B. Kumar, F. Alenezi, A. Alhudhaif, S. A. Althubiti, and K. Polat, "Automatic arrhythmia detection based on the probabilistic neural network with FPGA implementation," *Mathematical Problems in Engineering*, vol. 2022, Art. no. 7564036, 2022. 1, 2
- [5] D. Valencia, S. F. Fard, and A. Alimohammad, "An artificial neural network processor with a custom instruction set architecture for embedded applications," *IEEE Transactions on Circuits and Systems I: Regular Papers*, vol. 67, no. 12, pp. 5200–5210, 2020. 1, 2
- [6] S. Vinaykumar and R. Thilagavathy, "FPGA implementation of artificial neural network (ANN) for ECG signal classification," in *2022 IEEE International IoT, Electronics and Mechatronics Conference (IEMTRONICS)*, 2022. 1, 2
- [7] E. Won, "A hardware implementation of artificial neural networks using field programmable gate arrays," *Nuclear Instruments and Methods in Physics Research Section A*, vol. 581, pp. 816–820, 2007. 2